HPC0

Terminal, Editor, and a Little Programming

What This Worksheet Is About

The main purpose of this worksheet is to quickly verify that the HPC0 tools have been installed successfully.

Along the way, you will get a first introduction to:

- **working in the terminal,**
- **using a text editor,**
- **writing and running your first C program,**
- **writing and running your first ABC program.**

The goal is not to understand everything right away. Instead, you should become familiar with the basic development workflow:

write code → compile → run → fix errors

Step 1: The Terminal

Open a terminal and split it into two panes (left and right).

Goal:
- **Left:** editor (writing code)
- **Right:** compiling and running programs

This way, you can immediately see compiler error messages and fix them in the editor.

How do I split the terminal?

The exact key bindings depend on your system:
- **macOS (iTerm2)**
  - Split vertically: `Cmd` + `D`
  - Switch between panes: `Cmd` + `Shift` + `Arrow Key`
- **Linux / WSL (Konsole)**
  - Split vertically: `Ctrl` + `(`
  - Switch between panes: `Control` + `Shift` + `Arrow Key`

If these shortcuts do not work on your system, just let us know!

Step 2: Your First C Program with Nano

Switch to the left terminal pane. Type `nano hello.c` and press `Enter`. This opens the Nano editor. Enter the following program *(The coding style is intentional. If you are curious why it looks this way, just ask! We'll discuss coding style throughout the course—and "agree to disagree" is always an acceptable outcome)*

```c
#include <stdio.h>

int
main()
{
    printf("Hello, world!\n");
    return 0;
}
```

Save and exit:
- Save: `Ctrl` + `O`, then press `Enter`
- Exit: `Ctrl` + `X`

Afterwards, you are back at the terminal.

Step 3: Compiling Your First C Program

Compile the program
- Switch to the right terminal pane and type: `gcc hello.c`
- If *everything worked* correctly, *no error message* will be displayed.

Check the result
- Type: `ls`
- You should now see a new file called `a.out`. This is your compiled program.

Programm ausführen (1. Versuch)
- Type: `a.out`
- This will usually **not** work (feel free to ask me about *paths* 😉).

Run the program correctly
- Type: `./a.out`
- You should now see: `Hello, world!`

Why `./a.out`?
For security reasons, the terminal normally does **not** search the current directory when looking for programs to execute.
By typing `./`, you explicitly tell it: *Run the program located in the current directory.*

Experiment (Highly Recommended!)
Go back to the left terminal pane and intentionally introduce a syntax error into your program. For example, delete a `{`, `}`, or `;`.

Save the file and try to compile it again. Read the compiler's error message carefully. Can you infer from the error message what went wrong before just fixing the program by trial and error?

Step 4: Your First ABC Program with Nano

Switch to the left terminal pane. Type `nano hello.abc` and press `Enter`. This opens the Nano editor. Enter the following program:

```
@ <stdio.hdr>

fn main(): int
{
    printf("Hello, world!\n");
    return 0;
}
```

Save and exit:
- Save: `Ctrl` + `O`, then press `Enter`
- Exit: `Ctrl` + `X`

Afterwards, you are back at the terminal.

Compile and run the program
- Switch to the right terminal pane and type: `abc hello.abc`
- If everything worked correctly, no error message will be displayed.
- Run the program with: `./a.out`
- You should now see: `Hello, world!`

Step 5: Trying Another Editor

So far, you have been using Nano. It is easy to learn and a great choice for getting started. However, for larger projects, most developers eventually switch to a more powerful editor.

Alternatives

- **VS Code**

  Very user-friendly (so they say), with an excellent extension ecosystem. The downside is that you leave the pure terminal environment.

- **(Neo)Vim**

  Runs entirely inside the terminal and can be extremely efficient once you get used to it.

Throughout HPC0, I actively support **Neovim**. If you decide to learn it, you have **24/7** support from me.

To make getting started easier, everyone receives a **Vim Cheat Sheet**—both as a printed handout and as a digital copy. Whether you want it or not. 😉